Dev Ops Project-3Assignment of Reactjs E-commerce Application Deployment

Deploy the given React application to a production ready state

Prepared by: Md Rustam Ansari *Hcl Gui Batch:* DO-C-WE-E-B36

- *Project Scope URL & Refer the Guidelines:* https://docs.google.com/document/d/11kbFude1yL9C3r--VX-HM_bZYuWNusVI1P_mITJTDc4/edit

- *Project Tasks contents & Timeline*

SN.	Task Descriptions	Page	Duration
Phase 1:	Project Scope Summary and Guidelines	1-1	-- --
Phase 2:	Install Git bash using Winget	2-2	-- --
Phase 3:	AWS EC2 Infrastructure & Security Group	3-3	-- --
Phase 4:	Version Control (GitHub, Git Clone)	3-4	-- --
Phase 5:	Dockerization (Docker Hub & Git CLI)	4-6	-- --
Phase 6:	Bash Scripting (Write 2 Bash Script)	6-6	-- --
Phase 7:	Jenkins Configuration & Auto-Trigger Setup	7-7	-- --
Phase 8:	Open-Source Health Monitoring & Alerts	8-7	-- --
Phase 9:	Final Verification & Submission Checklist	6-6	-- --

- *Tech stacks need to be used:*
  - *AWS*
  - *Docker*
  - *DevOps*
  - *Version Control*

- *How do I submit my work?*
  - *Push all your work files to GitHub (O/P screenshot images must).*
  - *Submit your project to the version control public repository to verify them.*

- *Terms and Conditions?*
  - *Submit your project to the version control public repository to verify them.*
  - *You agree to not share this confidential document with anyone.*
  - *You agree to open-source your code (it may even look good on your profile!).*
  - *Do not mention our company's name anywhere in the code.We will never use your source code under any circumstances for any commercial purposes; this is just a basic assessment task.*

Follow the following workflow to perform assigned project accordingly

NOTE: Any violation of Terms and conditions is strictly prohibited. You are bound to adhere to it.

- *Phase 2: Install Git bash using Winget*

- *Create a project directory on local system / laptop*
- *Open PowerShell as Administrator and deploy Gitbsh / WDS / Linux:*

○ *Workflow Phase 2 (Install Git bash using Winget):*

PS D:\DevOps_Gui26> winget install --id Git.Git -e --source winget

```
PS D:\DevOps_Gui26> winget install --id Git.Git -e --source winget
Found Git [Git.Git] Version 2.56.0.3
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/git-for-windows/git/releases/download/v2.56.0.windows.3/Git-2.56.0.3-64-bit.exe
  ██████████████████████████████  63.7 MB / 63.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
```

○ *Phase 3: AWS EC2 Infrastructure & Security Group:*

- *Clone the below mentioned repository and deploy the application (Run application in port http:80).*
- *Repo URL: https://github.com/sriram-R-krishnan/devops-build*

○ *Workflow Phase 3 (Launch AWS Instance for access as Client):*

- *Project_Steps_1: Clone Repo for Application Deployment*
- *Launch an ec2 instance, access them using ssh (22 port), Install prerequisites and update.*
- Ensure AWS-CLI already installed and configure on laptop *and then clone the repo.*

$ aws --version

```
ubuntu@ip-172-31-18-3:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1014-aws source/x86_64.ubuntu.26
```

$ aws configure list

```
ubuntu@ip-172-31-18-3:~$ aws configure list
NAME       : VALUE                    : TYPE                 : LOCATION
profile    : <not set>                : None                 : None
access_key : ****************B4JY     : shared-credentials-file :
secret_key : ****************h+oB     : shared-credentials-file :
region     : ap-southeast-2           : config-file          : ~/.aws/config
```

- *Created a simple bash script to deploy ubuntu 26.04 aws ec2 instance with the default settings*

$ cat aws-ec2-instance.sh

```
md_rustam@DESKTOP-CPK0PUB:~$ cat aws-ec2-instance.sh
#!/usr/bin/env bash
set -e

KEY_NAME="keypair"
MY_IP="$(curl -s https://checkip.amazonaws.com)/32"

echo "1. Authorizing SSH ($MY_IP) on Default Security Group..."
SG_ID=$(aws ec2 describe-security-groups \
    --filters "Name=group-name,Values=default" \
    --query "SecurityGroups[0].GroupId" --output text)

# Ignore error if rule already exists from a previous run
aws ec2 authorize-security-group-ingress \
    --group-id "$SG_ID" --protocol tcp --port 22 --cidr "$MY_IP" 2>/dev/null || true

echo "2. Setting up SSH Key ($KEY_NAME.pem)..."
if [ ! -f "${KEY_NAME}.pem" ]; then
    aws ec2 create-key-pair --key-name "$KEY_NAME" \
        --query "KeyMaterial" --output text > "${KEY_NAME}.pem"
    chmod 400 "${KEY_NAME}.pem"
fi

echo "3. Launching Ubuntu 26.04 EC2 Instance..."
INSTANCE_ID=$(aws ec2 run-instances \
    --image-id resolve:ssm:/aws/service/canonical/ubuntu/server/26.04/stable/current/amd64/hvm/ebs-gp3/ami-id \
    --instance-type t3.micro \
    --key-name "$KEY_NAME" \
    --security-group-ids "$SG_ID" \
    --query "Instances[0].InstanceId" --output text)

echo "4. Waiting for instance ($INSTANCE_ID) to start..."
aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"

PUBLIC_IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
    --query "Reservations[0].Instances[0].PublicIpAddress" --output text)

echo "5. Waiting for SSH on $PUBLIC_IP..."
until ssh -i "${KEY_NAME}.pem" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=5 "ubuntu@${PUBLIC_IP}" exit 2>/dev/null; do
    sleep 3
done

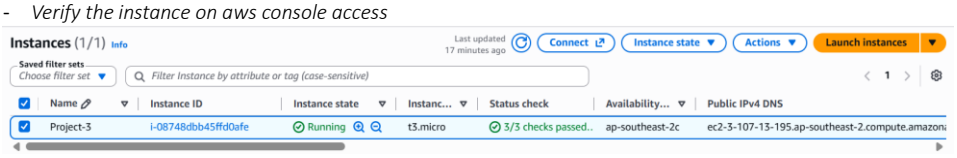
echo "Connected! (To reconnect later: ssh -i ${KEY_NAME}.pem ubuntu@${PUBLIC_IP})"
ssh -i "${KEY_NAME}.pem" "ubuntu@${PUBLIC_IP}"
```

- *The default security group typically does not allow inbound SSH (port 22), allow them.*

$ aws ec2 describe-instances --region ap-southeast-2 --query 'Reservations[*].Instances[*].[InstanceId,State.Name]' --output table

```
ubuntu@ip-172-31-18-3:~$ aws ec2 describe-instances   --region ap-southeast-2   --query 'Reservations[*].Instances[*].[InstanceId,State.Name]'   --output table
-----------------------------------
|        DescribeInstances        |
+---------------------+-----------+
|  i-08748dbb45ffd0afe |  running  |
+---------------------+-----------+
```

- *Verify the instance on aws console access*



- *Allowed inbound all traffic instead of "port 22, 80" in Security group / firewall.*



- *Update the latest OS patch, install prerequisite packages*
  $ sudo apt update -y; sudo apt upgrade -y

```
ubuntu@ip-172-31-18-3:~$ sudo apt update -y; sudo apt upgrade -y
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
All packages are up to date.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

○ *Phase 4: Version Control (GitHub, Git Clone)*

- *Dockerize the application by creating a Dockerfile*
- *Create a docker-compose file to use the above image*
- *Push the code to github to dev branch (use dockerignore & gitignore files)*
- *Note: Use only CLI for related git commands*

○ *Workflow Phase 4 (Create a New Repository on GitHub)*

- *Create a new empty repository on personal GitHub account*
- *Navigate the Project-3_React path and check the .gitignore and .dockerignore files*
- *Verify .gitignore and .dockerignore under the*

$ cat /home/ubuntu/Project-3_React/devops-build/.git

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ ls -la .git
total 48
drwxrwxr-x 7 ubuntu ubuntu 4096 Oct 25 01:08 .
drwxrwxr-x 4 ubuntu ubuntu 4096 Oct 25 01:08 ..
-rw-rw-r-- 1 ubuntu docker    0 Oct 25 00:24 .dockerignore
-rw-rw-r-- 1 ubuntu docker    0 Oct 25 00:23 .gitignore
```

- *Create a directory "Project-3_ React" into the ec2 instance and make git clone.*

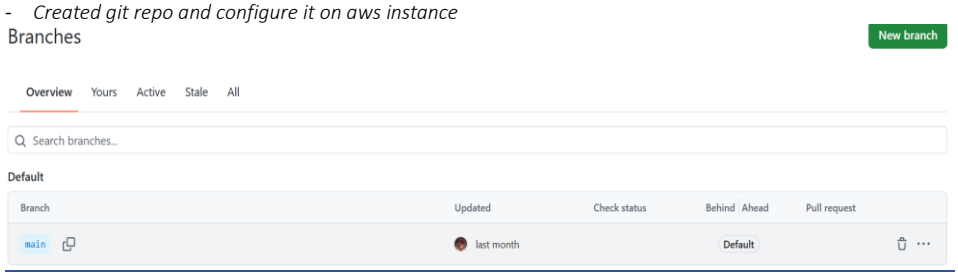
$ git clone https://github.com/sriram-R-krishnan/devops-build

```
ubuntu@ip-172-31-18-3:~$ mkdir Project-3_React; cd Project-3_React
ubuntu@ip-172-31-18-3:~/Project-3_React$ git clone https://github.com/sriram-R-krishnan/devops-build
Cloning into 'devops-build'...
remote: Enumerating objects: 21, done.
remote: Total 21 (delta 0), reused 0 (delta 0), pack-reused 21 (from 1)
Receiving objects: 100% (21/21), 720.09 KiB | 18.00 MiB/s, done.
```

- *Navigate to the cloned Git repository directory and list the contents of the directory and start the serve*

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build/build$ ls -l
total 48
-rw-rw-r-- 1 ubuntu ubuntu    18 Oct 24 21:35 _redirects
-rw-rw-r-- 1 ubuntu ubuntu   517 Oct 24 21:35 asset-manifest.json
-rw-rw-r-- 1 ubuntu ubuntu  3870 Oct 24 21:35 favicon.ico
-rw-rw-r-- 1 ubuntu ubuntu   961 Oct 24 21:35 index.html
-rw-rw-r-- 1 ubuntu ubuntu  5347 Oct 24 21:35 logo192.png
-rw-rw-r-- 1 ubuntu ubuntu  9664 Oct 24 21:35 logo512.png
-rw-rw-r-- 1 ubuntu ubuntu   492 Oct 24 21:35 manifest.json
-rw-rw-r-- 1 ubuntu ubuntu    67 Oct 24 21:35 robots.txt
drwxrwxr-x 4 ubuntu ubuntu  4096 Oct 24 21:35 static
```

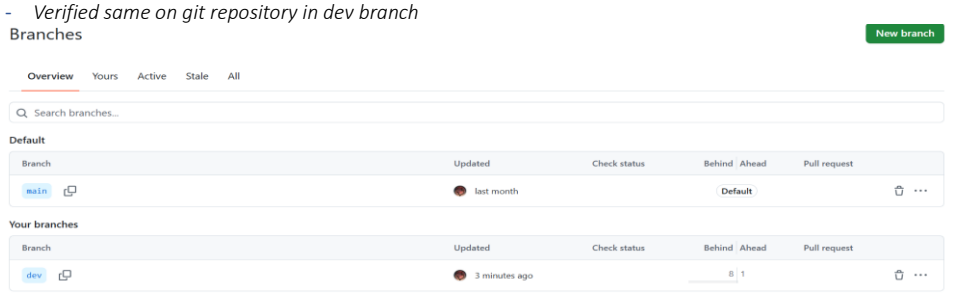
- *Created git repo and configure it on aws instance*



- *Push to the dev branch with the .gitignore and .dockerignore files*

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ git push -u origin dev
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 2 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (21/21), 720.09 KiB | 180.02 MiB/s, done.
Total 21 (delta 0), reused 21 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:      https://github.com/StarGithMd/Guvi_DevOps/pull/new/dev
remote:
To https://github.com/StarGithMd/Guvi_DevOps.git
 * [new branch]      dev -> dev
branch 'dev' set up to track 'origin/dev'.
```

- *Verified same on git repository in dev branch*



- *Point the cloned repo to your remote, create the dev and master branches, and push via CLI*

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ ls -la
total 44
drwxr-xr-x 4 md_rustam md_rustam  4096 Oct 25 05:50 .
drwxr-xr-x 3 md_rustam md_rustam  4096 Oct  4 07:42 ..
-rw-r--r-- 1 md_rustam md_rustam 12288 Oct  4 07:49 .Dockerfile.swp
drwxr-xr-x 7 md_rustam md_rustam  4096 Oct 25 05:53 .git
-rw-r--r-- 1 md_rustam md_rustam   300 Oct 25 05:49 Dockerfile
drwxr-xr-x 3 md_rustam md_rustam  4096 Oct  4 07:42 build
-rwxr-xr-x 1 md_rustam md_rustam   159 Oct 25 05:50 build.sh
-rwxr-xr-x 1 md_rustam md_rustam   241 Oct 25 05:50 deploy.sh
-rw-r--r-- 1 md_rustam md_rustam   164 Oct 25 05:49 docker-compose.yml
```

- *Also ensure master branch exists on remote*

$ git checkout -b master

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git checkout -b master
Switched to a new branch 'master'
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ ls -l
total 20
```

- *Stage and commit all newly created files*

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git add .
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git commit -m "Add Docker, Compose, and build/deploy automation scripts"
[dev 914bae3] Add Docker, Compose, and build/deploy automation scripts
 5 files changed, 42 insertions(+)
 create mode 100644 .Dockerfile.swp
 create mode 100644 Dockerfile
 create mode 100755 build.sh
 create mode 100755 deploy.sh
 create mode 100644 docker-compose.yml
```

- *Push to dev branch*

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git push -u origin dev
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.43 KiB | 734.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/StarGithMd/Guvi_DevOps.git
   ef44730..914bae3  dev -> dev
branch 'dev' set up to track 'origin/dev'.
```

○ *Phase 5: Docker_Containarization Application:*

- *Dockerize the application by creating a Dockerfile*
- *Create a docker-compose file to use the above image*
- *Create 2 repos "dev" and "prod" to push images.*
- *Prod" repo must be private and "dev" repo can be public*

○ *Workflow Phase 5 (Docker_Containarization):*

- *Project_Steps_2: Deploy prerequisite packages for the Containerization Application*
- *Update ubuntu and Install Docker, Docker Compose,*

$ sudo apt update && sudo apt install -y docker.io docker-compose-v2 git; sudo usermod -aG docker $USER newgrp docker

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ $ sudo apt update && sudo apt install -y docker.io docker-compose-v2 git
sudo usermod -aG docker $USER
newgrp docker
```

- *Create a multi-stage Dockerfile to build the React static files and serve them on port 80 via Nginx*

$ cat Dockerfile

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ cat Dockerfile
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with Nginx on Port 80
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- *Create docker-compose.yml using an environment variable so it works seamlessly with both dev and prod images*

$ cat docker-compose.yml

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ $ cat > docker-compose.yml << EOF
version: '3.8'
services:
  react-app:
    image: ${IMAGE_NAME:-<your-dockerhub-username>/dev:latest}
    container_name: react-app
    ports:
      - "80:80"
    restart: always
EOF
```

- *Create a .gitignore file to exclude local artifacts*

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ $ cat > .gitignore << EOF
node_modules
build
.env
*.log
.DS_Store
EOF
```

- *Create a .dockerignore file to keep the Docker build context lightweight*

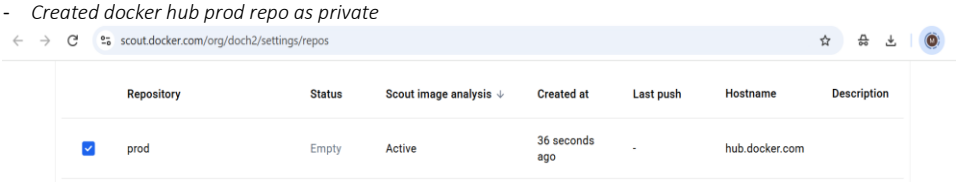
```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ $ cat > .dockerignore << EOF
node_modules
build
.git
.gitignore
Dockerfile
docker-compose.yml
EOF
```

- *Create Prod" repo must be private and "dev" repo can be public*
- *Ensure docker.io should be Installed and other required packages as well.*

$ sudo apt install docker.io

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ sudo apt install docker.io
docker.io is already the newest version (29.1.3-0ubuntu4.1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- *Created docker hub prod repo as private*



- *Login docker repository*

$ docker login-u doch2

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ docker login -u doch2

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/home/md_rustam/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

- Verify the docker hub existing repository

$ docker tag project3-app:latest doch2/dev:latest

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build/build$ docker push doch2/dev:latest
The push refers to repository [docker.io/doch2/dev]
d9aae54b5831: Pushed
ab7ff2010ee1: Pushed
e2de96513ba9: Pushed
44136fa355b3: Mounted from jenkins/jenkins
6c53d0b2a666: Pushed
745dfb2690dd: Pushed
9a9a644fdd6a: Pushed
64c8194480fe: Pushed
e76228b47809: Pushed
e72112c14215: Pushed
2a697d3cc502: Pushed
latest: digest: sha256:ea32490c50aacf4a3ce767194b1f5f09e50107e6beb6ee792564dd105f88eb5b size: 856
```

- *Merge Dev to Master*

$ git checkout master; git merge dev; git origin master

```
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git checkout master
Already on 'master'
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git merge dev
Already up to date.
md_rustam@DESKTOP-CPK0PUB:~/Final_Project-3/devops-build$ git push origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/StarGithMd/Guvi_DevOps/pull/new/master
remote:
To https://github.com/StarGithMd/Guvi_DevOps.git
 * [new branch]      master -> master
```

○ *Phase 6: Bash Scripting (Write 2 Bash Script)*

- *build.sh for building docker images*
- *deploy.sh for deploying the image to server*

○ *Workflow Phase 6 (Create Write 2 Bash Script)*

Commented [M.1]:

- *Write 2 Bash Script (build.sh and deploy.sh)*
- *Create Bash Scripts (Create build.sh to build the Docker image dynamically)*

$ cat build.sh

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ cat > build.sh << EOF
#!/bin/bash
set -e

IMAGE_REPO=${1:-"<your-dockerhub-username>/dev"}
IMAGE_TAG=${2:-"latest"}

echo "Building Docker image: ${IMAGE_REPO}:${IMAGE_TAG}..."
docker build -t ${IMAGE_REPO}:${IMAGE_TAG} .
echo "Build complete!"
EOF
```

- Create deploy.sh to pull and deploy the container via Docker Compose

$ cat deploy.sh

```
ubuntu@ip-172-31-18-3:~/Project-3_React/devops-build$ cat > deploy.sh << EOF
#!/bin/bash
set -e

export IMAGE_NAME=${1:-"<your-dockerhub-username>/dev:latest"}

echo "Deploying container with image: ${IMAGE_NAME}..."
docker compose down || true
docker compose pull
docker compose up -d
echo "Application successfully deployed on port 80!"
EOF
```

- *Make both scripts executable*

$ chmod +x build.sh deploy.sh

o *Phase 7: Jenkins (Install and configure jenkins build)*

- *Install and configure jenkins build step as per needs to build, push & deploy the application*
- *Connect jenkins to the github repo with auto build trigger from both dev & master branch*
- *If code pushed to dev branch, docker image must build and pushed to dev repo in docker hub*
- *If dev merged to master, then docker image must be pushed to prod repo in docker hub*

o *Workflow Phase 7 (Installation and configure Jenkins build).*

- *Install and configure jenkins build step as per needs to build, push & deploy the application*
- *Launch new AWS Ec2 Instance to deploy Jenkins*

$ bash aws-ec2-instance.sh

```
md_rustam@DESKTOP-CPK0PUB:~$ ./aws-ec2-instance.sh
1. Authorizing SSH (103.95.166.30/32) on Default Security Group...
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-0277d458721af3529",
            "GroupId": "sg-0c7439b8f5653a0de",
            "GroupOwnerId": "633697460172",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "CidrIpv4": "103.95.166.30/32",
            "SecurityGroupRuleArn": "arn:aws:ec2:ap-southeast-2:633697460172:security-group-rule/sgr-0277d458721af3529"
        }
    ]
}
2. Setting up SSH Key (keypair.pem)...
3. Launching Ubuntu 26.04 EC2 Instance...
4. Waiting for instance (i-0404ebc287184b8cd) to start...
5. Waiting for SSH on 13.211.146.74...
Connected! (To reconnect later: ssh -i keypair.pem ubuntu@13.211.146.74)
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Oct 22 05:19:08 UTC 2026

  System load:    1.22      Processes:             23
  Usage of /home: unknown   Users logged in:       0
  Memory usage:   3%        IPv4 address for eth0: 10.10.10.2
  Swap usage:     0%


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


ubuntu@ip-172-31-47-218:~$ sudo apt update
```

- *Install Jenkins on newly launched ec2 instance*

$ java -version

```
ubuntu@ip-172-31-47-218:~$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-26.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-26.04.4-Ubuntu, mixed mode, sharing)
```

$ sudo apt update; sudo apt install openjdk-21-jdk -y

```
ubuntu@ip-172-31-47-218:~$ sudo apt update

sudo apt install openjdk-21-jdk -y

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins -y

sudo systemctl enable jenkins

sudo systemctl start jenkins
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:4 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]
Get:5 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/universe Translation-en [6329 kB]
Get:6 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 Components [4556 kB]
Get:7 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 c-n-f Metadata [313 kB]
Get:8 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/multiverse amd64v3 Packages [291 kB]
Get:9 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/multiverse Translation-en [127 kB]
Get:10 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute/multiverse amd64 Components [50.0 kB]
```

- *Import the Jenkins GPG Key*

*$ curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | *
sudo gpg --dearmor -o /usr/share/keyrings/jenkins-keyring.gpg

```
ubuntu@ip-172-31-47-218:~$ curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \
sudo gpg --dearmor -o /usr/share/keyrings/jenkins-keyring.gpg
```

- *Add the Jenkins Repository*

$ echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.gpg] https://pkg.jenkins.io/debian-stable binary/" | \ sudo tee /etc/apt/sources.list.d/jenkins.list

```
ubuntu@ip-172-31-47-218:~$ echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.gpg] https://pkg.jenkins.io/debian-stable binary/" | \
sudo tee /etc/apt/sources.list.d/jenkins.list
deb [signed-by=/usr/share/keyrings/jenkins-keyring.gpg] https://pkg.jenkins.io/debian-stable binary/
```

- *$ systemctl status jenkins*

```
ubuntu@ip-172-31-47-218:~$ systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-25 04:00:39 UTC; 1min 53s ago
 Invocation: c40e8b456e3c40dd801af4b8c5b2a5c6
   Main PID: 599 (java)
      Tasks: 37 (limit: 627)
     Memory: 295.9M (peak: 297.1M)
        CPU: 17.440s
     CGroup: /system.slice/jenkins.service
             └─599 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF]>
Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF] This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF]>
Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF] ************************************************************
Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF] ************************************************************
Oct 25 04:00:33 ip-172-31-47-218 jenkins[599]: [LF] ************************************************************
Oct 25 04:00:39 ip-172-31-47-218 jenkins[599]: 2026-10-25 04:00:39.619+0000 [id=31]       INFO       jenkins.InitReactorRunner$1#onAttained: Completed initialization
Oct 25 04:00:39 ip-172-31-47-218 jenkins[599]: 2026-10-25 04:00:39.658+0000 [id=24]       INFO       hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Oct 25 04:00:39 ip-172-31-47-218 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 25 04:00:44 ip-172-31-47-218 jenkins[599]: 2026-10-25 04:00:44.749+0000 [id=63]       WARNING       h.n.DiskSpaceMonitorDescriptor#markNodeOfflineOrOnline: Making Built-In
```

$ docker volume create jenkins_home

```
ubuntu@ip-172-31-47-218:~$ docker volume create jenkins_home
jenkins_home
```

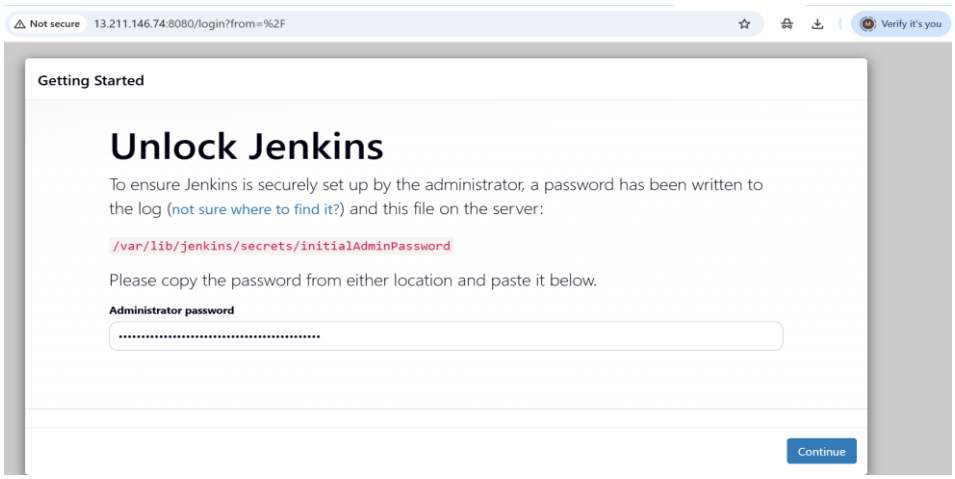
$ systemctl status docker; sudo usermod -aG docker ubuntu; newgrp docker

```
ubuntu@ip-172-31-47-218:~$  systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-10-25 04:00:27 UTC; 9min ago
 Invocation: 378f0c73d27d40df8d2a08006bac0e20
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 936 (dockerd)
      Tasks: 9
     Memory: 92.8M (peak: 93.2M)
        CPU: 612ms
     CGroup: /system.slice/docker.service
             └─936 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

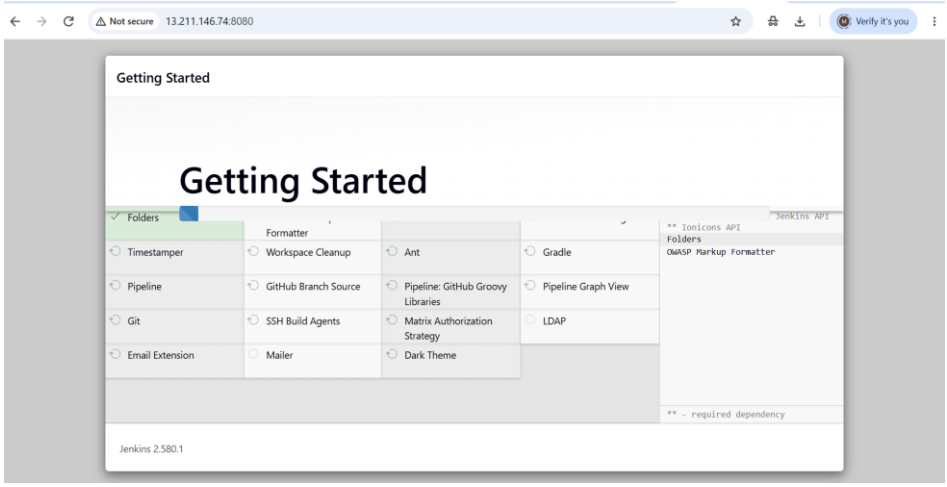
$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword

```
ubuntu@ip-172-31-47-218:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
0ed25ea3561e4ad094c7e4e07c6b96bf
```

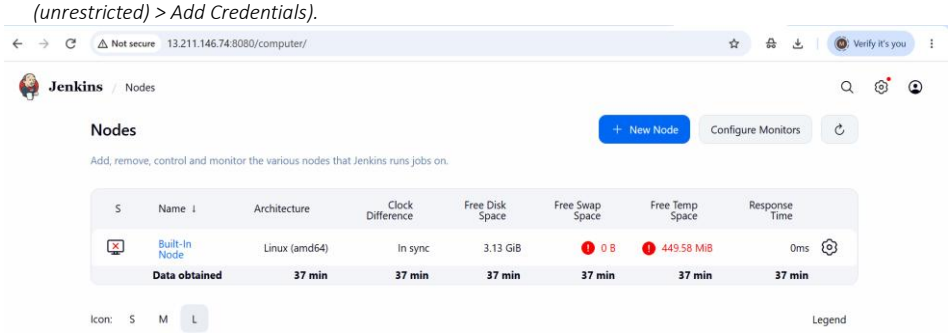
$ http://13.211.146.74:8080/login?from=%2F



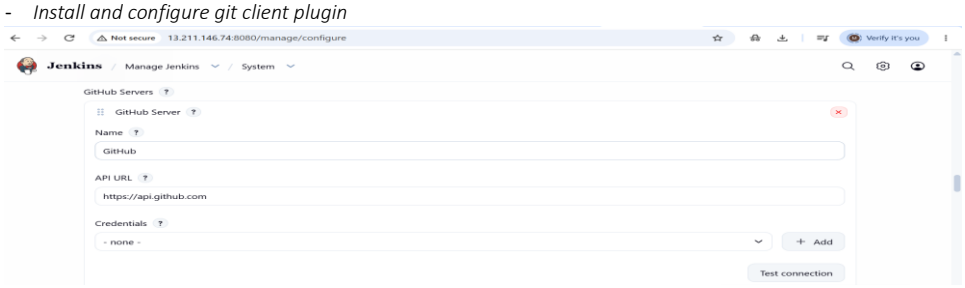
- *Install Required Jenkins Plugins (Git, GitHub, GitHub Branch Source, Pipeline, Docker, Docker Pipeline, Credentials Binding, Multibranch Pipeline)*



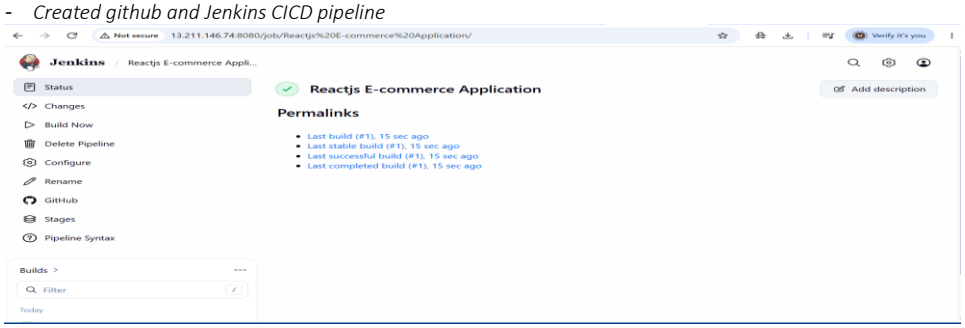
- *Configure Credentials (Navigate to Manage Jenkins > Credentials > System > Global credentials (unrestricted) > Add Credentials).*



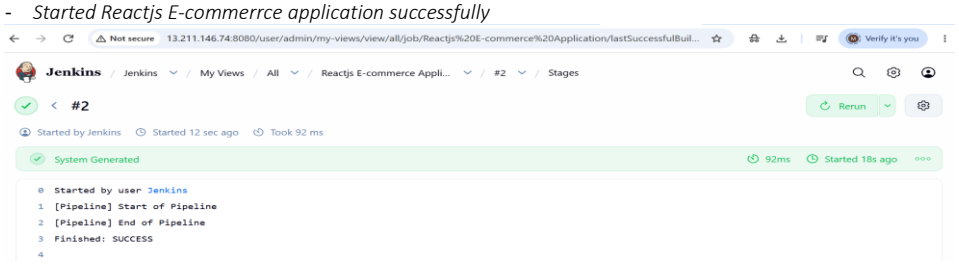
- *Install and configure git client plugin*



- *Created github and Jenkins CICD pipeline*

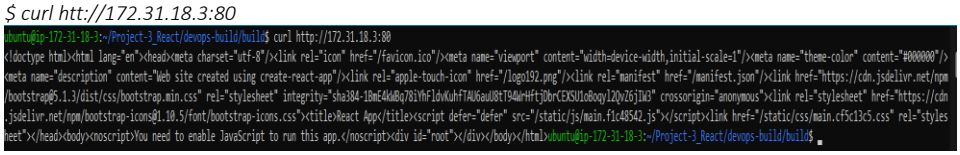


- *Started Reactjs E-commerrce application successfully*



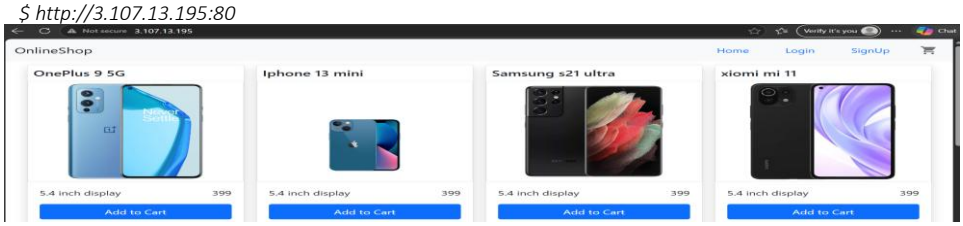
- *Verify access through the CLI*

$ curl htt://172.31.18.3:80



- *Verify the Reactjs E-commerce application through the browser with public ip address*

$ http://3.107.13.195:80



Phase 8: Open-Source Health Monitoring & Alerts

- *This setup is widely used in DevOps environments and supports email notifications when the application goes down.*
- *Lightweight Monitoring with Uptime Kuma (Recommended for Projects)*
- *Enterprise Monitoring with Prometheus*

Workflow Phase 7 (Open-Source Health Monitoring & Alerts)

- *Open-Source Monitoring & Alerting Setup*
- ***Uptime Kuma** is a lightweight, self-hosted open-source monitoring tool*
- *that tracks HTTP health status and sends instant alerts (Slack, Email, Telegram, Discord) strictly when the application goes down.*
- *Run Uptime Kuma on your EC2 instance:*

```
docker run -d \
  --restart=always \
  -p 3001:3001 \
  -v uptime-kuma:/app/data \
  --name uptime-kuma \
  louislam/uptime-kuma:1
```

- *Open http:// 3.107.13.195:3001 and create an admin account.*
- *Click Add New Monitor:*
  - *Monitor Type: HTTP(s)*
  - *Friendly Name: React Production App*
  - *URL: http:// 3.107.13.195:80*
  - *Heartbeat Interval: 60 seconds*

- *Click Setup Notification (on the right panel):*
  - *Select your preferred provider (e.g., Email (SMTP), Slack, Telegram, or Discord).*
  - *Configure the webhook/SMTP details, test the alert, and click Save.*
  - *Uptime Kuma will now trigger notifications only when the container on port 80 stops responding.*

Phase 8: Final Verification & Submission Checklist

Workflow Phase 7 (Final Verification & Submission Checklist)

- *Create a screenshots/ folder or update README.md in your GitHub repository with the required submission details and screenshots.*
- *git add .; git commit -m "19_Monitoring_Task_1"; git push origin main*

Thanks